

```
In [ ]: import numpy as np

def fun1():
    lst = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
    print(lst)

    arr1 = np.array(lst)
    print(arr1)
    print(type(arr1)) # <class 'numpy.ndarray'>
    print(arr1.shape) # (10,)
    print(arr1.dtype) # int64
    print(arr1.size)
    print(arr1.ndim)   # 1
    print(arr1.itemsize)

    arr2 = np.array([[0, 1, 2], [3, 4, 5], [6, 7, 8]])
    print(arr2)
    print(type(arr2)) # <class 'numpy.ndarray'>
    print(arr2.shape) # (3, 3)
    print(arr2.dtype) # int64
    print(arr2.size) # 9
    print(arr2.ndim)   # 2
    print(arr2.itemsize)

def fun2():
    lst = [i for i in range(0,25)]
    arr = np.array(lst)
    print(arr.shape)
    arr.shape = (5, 5)
    print(arr)
    print(arr.shape)

if __name__ == "__main__":
    #fun1()
    fun2()
```

```
(25,)
[[ 0  1  2  3  4]
 [ 5  6  7  8  9]
 [10 11 12 13 14]
 [15 16 17 18 19]
 [20 21 22 23 24]]
(5, 5)
```

```
In [22]: import numpy as np # 导入 NumPy 库，通常用 np 作为别名

# ===== 1. 数组创建 =====
# NumPy 提供了多种创建数组的方式，适应不同场景需求
print("=" * 50)
print("1. 数组创建")
print("=" * 50)

# --- np.array(): 从 Python 列表或元组创建数组 ---
# 这是最基础的创建方式，将 Python 序列转换为 ndarray 对象
a = np.array([1, 2, 3, 4, 5])
print("从列表创建:", a)
```

```

# --- np.zeros(shape): 创建全 0 数组 ---
# 参数 shape 是一个元组, 指定数组各维度大小
# 常用于初始化占位数组
print("zeros(3,4):\n", np.zeros((3, 4)))

# --- np.ones(shape): 创建全 1 数组 ---
# 与 zeros 类似, 但所有元素初始化为 1
print("ones(2,3):\n", np.ones((2, 3)))

# --- np.full(shape, fill_value): 创建填充指定值的数组 ---
# 第一个参数是形状, 第二个参数是要填充的常量值
print("full((2,2), 7):\n", np.full((2, 2), 7))

# --- np.eye(N): 创建 N×N 的单位矩阵 (对角线为 1, 其余为 0) ---
# 在线性代数中非常常用
print("eye(3):\n", np.eye(3))

# --- np.arange(start, stop, step): 创建等差数组 ---
# 类似 Python 的 range(), 但返回 ndarray, 支持浮点数步长
# 包含 start, 不包含 stop
print("arange(0,10,2):", np.arange(0, 10, 2))

# --- np.linspace(start, stop, num): 创建等间隔数组 ---
# 在 [start, stop] 之间均匀取 num 个点 (包含两端点)
# 常用于生成绘图用的 x 轴坐标
print("linspace(0,1,5):", np.linspace(0, 1, 5))

# --- 随机数组 ---
# np.random.seed(): 固定随机种子, 保证每次运行结果一致 (调试必备)
np.random.seed(42)

# np.random.rand(d0, d1, ...): 生成 [0, 1) 区间均匀分布的随机数
print("random.rand(3,2):\n", np.random.rand(3, 2))

# np.random.randint(low, high, size): 生成指定范围的随机整数
# low: 下界 (包含), high: 上界 (不包含), size: 形状
print("random.randint(0,10,(3,3)):\n", np.random.randint(0, 10, (3, 3)))

# np.random.randn(d0, d1, ...): 生成标准正态分布 N(0,1) 的随机数
print("random.randn(2,3) (正态分布):\n", np.random.randn(2, 3))

# ===== 2. 数组属性 =====
# ndarray 对象有许多属性用于描述其结构信息
print("\n" + "=" * 50)
print("2. 数组属性")
print("=" * 50)

arr = np.random.randint(0, 100, (3, 4))
print("arr:\n", arr)
print("shape:", arr.shape)      # shape: 各维度大小, 返回元组, 如 (3,4) 表示 3
print("ndim:", arr.ndim)        # ndim: 数组的维度数 (几维数组)
print("size:", arr.size)        # size: 数组元素总个数 = 各维度乘积
print("dtype:", arr.dtype)      # dtype: 元素的数据类型, 如 int32, float64 等
print("itemsize:", arr.itemsize, "bytes") # itemsize: 每个元素占用的字节数

# ===== 3. 索引与切片 =====
# NumPy 的索引从 0 开始, 支持类似 Python 列表的切片语法
print("\n" + "=" * 50)
print("3. 索引与切片")
print("=" * 50)

```

```

arr = np.arange(1, 21).reshape(4, 5) # 创建 1~20 的数组并重塑为 4x5
print("arr (4x5):\n", arr)

# 单个元素索引: arr[行, 列]
print("arr[0, 0]:", arr[0, 0]) # 第 1 行第 1 列

# 行切片: 取整行
print("arr[2, :]:", arr[2, :]) # 取第 3 行所有列

# 列切片: 取整列
print("arr[:, 3]:", arr[:, 3]) # 取所有行的第 4 列

# 子矩阵切片: arr[行范围, 列范围], 左闭右开
print("arr[1:3, 1:4]:\n", arr[1:3, 1:4]) # 取第2~3行、第2~4列

# --- 花式索引 (Fancy Indexing) ---
# 用整数列表指定要选取的行/列, 不要求连续
print("arr[[0,2,3]]:\n", arr[[0, 2, 3]]) # 取第1、3、4行 (按列表顺序排列)

# ===== 4. 布尔索引 =====
# 使用布尔数组作为掩码来筛选满足条件的元素, 是数据分析的利器
print("\n" + "=" * 50)
print("4. 布尔索引")
print("=" * 50)

arr = np.random.randint(0, 50, (3, 4))
print("arr:\n", arr)

# 比较运算生成布尔数组, 形状与原数组相同
print("arr > 25:\n", arr > 25)

# 用布尔数组直接索引, 返回所有 True 位置的元素 (一维数组)
print("arr[arr > 25]:", arr[arr > 25])

# 组合条件: 用 & (与)、| (或)、~ (非) 连接, 每个条件需加括号
print("arr[(arr > 10) & (arr < 30)]:", arr[(arr > 10) & (arr < 30)])

# ===== 5. 变形与拼接 =====
# 改变数组形状、合并多个数组是数据预处理中的常见操作
print("\n" + "=" * 50)
print("5. 变形与拼接")
print("=" * 50)

arr = np.arange(12) # 创建 0~11 的一维数组
print("原始:", arr)

# reshape(new_shape): 改变形状, 总元素数必须匹配
print("reshape(3,4):\n", arr.reshape(3, 4)) # 变成 3x4 二维数组
print("reshape(2,2,3):\n", arr.reshape(2, 2, 3)) # 变成 2x2x3 三维数组

# ravel(): 将多维数组展平为一维数组 (返回视图, 不复制数据)
print("ravel() 展平:", arr.reshape(3, 4).ravel())

# --- 数组拼接 ---
a = np.array([[1, 2], [3, 4]])
b = np.array([[5, 6]])

# vstack(): 垂直堆叠 (沿第 0 轴, 增加行数)
print("vstack:\n", np.vstack([a, b]))

```

```

# hstack(): 水平堆叠 (沿第 1 轴, 增加列数)
# b.T 是 b 的转置, 使列数匹配
print("hstack:\n", np.hstack([a, b.T]))

# concatenate(): 沿指定轴拼接, 更通用的拼接函数
print("concatenate(axis=0):\n", np.concatenate([a, b], axis=0))

# ===== 6. 数学运算 =====
# NumPy 支持向量化运算, 对数组逐元素操作, 比 Python 循环快数百倍
print("\n" + "=" * 50)
print("6. 数学运算")
print("=" * 50)

a = np.array([1, 2, 3, 4])
b = np.array([10, 20, 30, 40])
print("a:", a, " b:", b)

# 逐元素四则运算
print("a + b:", a + b)    # 加法
print("a * b:", a * b)    # 逐元素乘法 (不是矩阵乘法)
print("a ** 2:", a ** 2)  # 逐元素幂运算

# 通用函数 (ufunc): 对数组逐元素应用的数学函数
print("np.sqrt(a):", np.sqrt(a)) # 平方根
print("np.exp(a):", np.exp(a))   # e^x
print("np.log(a):", np.log(a))   # 自然对数 ln(x)
print("np.sin(a):", np.sin(a))   # 正弦函数

# ===== 7. 广播 (Broadcasting) =====
# 广播是 NumPy 的核心机制之一: 让不同形状的数组进行运算
# 规则: 从末尾维度向前比较, 维度相等或其中一个为 1 即可广播
print("\n" + "=" * 50)
print("7. 广播 (Broadcasting)")
print("=" * 50)

a = np.array([[1, 2, 3], [4, 5, 6]]) # shape (2, 3)
b = np.array([10, 20, 30])           # shape (3,)

# b 会自动「扩展」为 (2, 3) 再与 a 逐元素相加
print("a:\n", a)
print("b:", b)
print("a + b (广播):\n", a + b)

# 列向量广播: shape (2, 1) 与 (2, 3) → b 沿列方向扩展
c = np.array([[1], [2]])             # shape (2, 1)
print("a + c (广播):\n", a + c)

# ===== 8. 聚合函数 =====
# 对数组进行统计汇总, 可指定轴方向
print("\n" + "=" * 50)
print("8. 聚合函数")
print("=" * 50)

arr = np.random.randint(1, 100, (3, 4))
print("arr:\n", arr)

# 全局聚合 (不指定轴, 对所有元素操作)
print("sum:", arr.sum())              # 所有元素求和
print("mean:", arr.mean())            # 平均值

```

```

print("std:", arr.std()) # 标准差
print("max:", arr.max(), " min:", arr.min()) # 最大值 / 最小值
print("argmax:", arr.argmax(), " argmin:", arr.argmin()) # 最大/最小值的索引 (

# 按轴聚合: axis=0 沿行方向 (对每列操作), axis=1 沿列方向 (对每行操作)
print("按行求和 (axis=1):", arr.sum(axis=1)) # 每一行的和
print("按列求均值 (axis=0):", arr.mean(axis=0)) # 每一列的均值

# cumsum(): 累计和, 沿指定轴逐步累加
print("cumsum:\n", arr.cumsum(axis=1))

# ===== 9. 排序与唯一值 =====
print("\n" + "=" * 50)
print("9. 排序与唯一值")
print("=" * 50)

arr = np.random.randint(0, 10, 10)
print("原始:", arr)

# np.sort(): 返回排序后的新数组, 原数组不变
print("np.sort:", np.sort(arr))

# np.unique(): 返回去重并排序后的唯一值
print("np.unique:", np.unique(arr))

# np.bincount(): 统计每个非负整数出现的次数
# 索引表示数值, 值表示该数值出现的次数
print("np.bincount:", np.bincount(arr))

# ===== 10. 线性代数 =====
# NumPy 的 linalg 模块提供了丰富的线性代数运算
print("\n" + "=" * 50)
print("10. 线性代数")
print("=" * 50)

A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])
print("A:\n", A)
print("B:\n", B)

# 矩阵乘法 (两种等价写法)
print("A @ B (矩阵乘法):\n", A @ B) # Python 3.5+ 的 @ 运算符, 推荐
print("np.dot(A, B):\n", np.dot(A, B)) # 传统写法

# 矩阵转置: 行列互换
print("A.T (转置):\n", A.T)

# 逆矩阵: A @ A_inv = I (单位矩阵)
print("np.linalg.inv(A) (逆矩阵):\n", np.linalg.inv(A))

# 行列式: 反映矩阵的缩放因子
print("np.linalg.det(A) (行列式):", np.linalg.det(A))

# 解线性方程组 Ax = b, 即求 x 使得 A @ x = b
b_vec = np.array([5, 11])
x = np.linalg.solve(A, b_vec)
print("解 Ax = [5,11]: x =", x)

# 特征值与特征向量: A @ v = λ @ v
print("np.linalg.eig(A) (特征值/特征向量):")

```

```
eigenvalues, eigenvectors = np.linalg.eig(A)
print(" 特征值:", eigenvalues)      # λ
print(" 特征向量:\n", eigenvectors) # 每列是一个特征向量

# ===== 11. 条件选择 =====
# 根据条件从数组中选择或限制元素值
print("\n" + "=" * 50)
print("11. 条件选择")
print("=" * 50)

arr = np.arange(10)
print("arr:", arr)

# np.where(condition, x, y): 条件为 True 取 x, False 取 y
# 相当于向量化的 if-else
print("np.where(arr > 5, 1, 0):", np.where(arr > 5, 1, 0))

# np.clip(arr, min, max): 将数组值限制在 [min, max] 区间内
# 小于 min 的变为 min, 大于 max 的变为 max
print("np.clip(arr, 3, 7):", np.clip(arr, 3, 7))

# ===== 12. 文件读写 =====
# NumPy 支持高效的数组序列化存储
print("\n" + "=" * 50)
print("12. 文件读写")
print("=" * 50)

arr = np.arange(12).reshape(3, 4)

# --- 二进制格式 (.npy) ---
# 保存为 NumPy 专用二进制格式, 读写速度快, 保留 dtype 信息
np.save('numpy_demo.npy', arr)
loaded = np.load('numpy_demo.npy')
print("保存并读取 (.npy):\n", loaded)

# --- 文本格式 (.csv) ---
# 保存为人类可读的 CSV 文本文件
# fmt='%d' 指定整数格式, delimiter=',' 指定分隔符
np.savetxt('numpy_demo.csv', arr, delimiter=',', fmt='%d')
loaded_csv = np.loadtxt('numpy_demo.csv', delimiter=',')
print("CSV格式读取:\n", loaded_csv)

print("\n✅ NumPy 功能演示完成!")
```

```
=====
1. 数组创建
=====
从列表创建: [1 2 3 4 5]
zeros(3,4):
[[0. 0. 0. 0.]
 [0. 0. 0. 0.]
 [0. 0. 0. 0.]]
ones(2,3):
[[1. 1. 1.]
 [1. 1. 1.]]
full((2,2), 7):
[[7 7]
 [7 7]]
eye(3):
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
arange(0,10,2): [0 2 4 6 8]
linspace(0,1,5): [0.  0.25 0.5  0.75 1.  ]
random.rand(3,2):
[[0.37454012 0.95071431]
 [0.73199394 0.59865848]
 [0.15601864 0.15599452]]
random.randint(0,10,(3,3)):
[[7 4 3]
 [7 7 2]
 [5 4 1]]
random.randn(2,3) (正态分布):
[[ 0.23309524  0.11799461  1.46237812]
 [ 1.53871497 -2.43910582  0.60344123]]

=====
2. 数组属性
=====
arr:
[[91 59 79 14]
 [61 61 46 61]
 [50 54 63  2]]
shape: (3, 4)
ndim: 2
size: 12
dtype: int32
itemsize: 4 bytes

=====
3. 索引与切片
=====
arr (4x5):
[[ 1  2  3  4  5]
 [ 6  7  8  9 10]
 [11 12 13 14 15]
 [16 17 18 19 20]]
arr[0, 0]: 1
arr[2, :]: [11 12 13 14 15]
arr[:, 3]: [ 4  9 14 19]
arr[1:3, 1:4]:
[[ 7  8  9]
 [12 13 14]]
arr[[0,2,3]]:
```

```
[[ 1  2  3  4  5]
 [11 12 13 14 15]
 [16 17 18 19 20]]
```

4. 布尔索引

```
arr:
[[36  6 20  8]
 [38 17  3 24]
 [13 49  8 25]]
arr > 25:
[[ True False False False]
 [ True False False False]
 [False  True False False]]
arr[arr > 25]: [36 38 49]
arr[(arr > 10) & (arr < 30)]: [20 17 24 13 25]
```

5. 变形与拼接

```
原始: [ 0  1  2  3  4  5  6  7  8  9 10 11]
reshape(3,4):
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
reshape(2,2,3):
[[[ 0  1  2]
   [ 3  4  5]]

  [[ 6  7  8]
   [ 9 10 11]]]
ravel() 展平: [ 0  1  2  3  4  5  6  7  8  9 10 11]
vstack:
[[1 2]
 [3 4]
 [5 6]]
hstack:
[[1 2 5]
 [3 4 6]]
concatenate(axis=0):
[[1 2]
 [3 4]
 [5 6]]
```

6. 数学运算

```
a: [1 2 3 4]   b: [10 20 30 40]
a + b: [11 22 33 44]
a * b: [ 10  40  90 160]
a ** 2: [ 1  4  9 16]
np.sqrt(a): [1.         1.41421356 1.73205081 2.         ]
np.exp(a): [ 2.71828183  7.3890561  20.08553692 54.59815003]
np.log(a): [0.         0.69314718 1.09861229 1.38629436]
np.sin(a): [ 0.84147098  0.90929743  0.14112001 -0.7568025 ]
```

7. 广播 (Broadcasting)

```

a:
[[1 2 3]
 [4 5 6]]
b: [10 20 30]
a + b (广播):
[[11 22 33]
 [14 25 36]]
a + c (广播):
[[2 3 4]
 [6 7 8]]

=====
8. 聚合函数
=====
arr:
[[53  2 84 92]
 [60 71 44  8]
 [47 35 78 81]]
sum: 655
mean: 54.583333333333336
std: 27.84019376528994
max: 92   min: 2
argmax: 3   argmin: 1
按行求和 (axis=1): [231 183 241]
按列求均值 (axis=0): [53.33333333 36.        68.66666667 60.33333333]
cumsum:
[[ 53  55 139 231]
 [ 60 131 175 183]
 [ 47  82 160 241]]

=====
9. 排序与唯一值
=====
原始: [3 1 7 3 1 5 5 9 3 5]
np.sort: [1 1 3 3 3 5 5 5 7 9]
np.unique: [1 3 5 7 9]
np.bincount: [0 2 0 3 0 3 0 1 0 1]

=====
10. 线性代数
=====
A:
[[1 2]
 [3 4]]
B:
[[5 6]
 [7 8]]
A @ B (矩阵乘法):
[[19 22]
 [43 50]]
np.dot(A, B):
[[19 22]
 [43 50]]
A.T (转置):
[[1 3]
 [2 4]]
np.linalg.inv(A) (逆矩阵):
[[-2.   1.]
 [ 1.5 -0.5]]
np.linalg.det(A) (行列式): -2.0000000000000004

```

```
解 Ax = [5,11]: x = [1. 2.]
np.linalg.eig(A) (特征值/特征向量):
  特征值: [-0.37228132  5.37228132]
  特征向量:
    [[-0.82456484 -0.41597356]
     [ 0.56576746 -0.90937671]]
```

11. 条件选择

```
arr: [0 1 2 3 4 5 6 7 8 9]
np.where(arr > 5, 1, 0): [0 0 0 0 0 0 1 1 1 1]
np.clip(arr, 3, 7): [3 3 3 3 4 5 6 7 7 7]
```

12. 文件读写

保存并读取 (.npz):

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

CSV格式读取:

```
[[ 0.  1.  2.  3.]
 [ 4.  5.  6.  7.]
 [ 8.  9. 10. 11.]]
```

✅ NumPy 功能演示完成!